Using Model Free Reinforcement Learning to Find Optimal Policies to Guide a Player Across a Frozen Lake Environment

Connor P. Gordon*
University of Illinois at Urbana-Champaign, Urbana, IL, 61801

This report explores using the model-free methods of on-policy first-visit Monte Carlo Control, SARSA, and Q-Learning to find the optimal policy for the simple Frozen Lake MDP.

I. Introduction

Markov Decision Processes, often shortened to "MDPs", are used to model many complex, real-world problems. For example, the driving and maneuvers of an autonomous vehicle can be modeled using an MDP. These models are useful because they enable the optimization of difficult and seemingly random problems by a policy, π. One of the most important aspects and goals of the reinforcement learning field is the optimization of this policy. Within the scope of finding the optimal policy for MDPs, many methods can be used. Simple methods such as *Value Iteration* and *Policy Iteration* can be used when the MDP is simple and the transition and reward functions are known. However, for more complex MDPs where the exact environment dynamics are not known, model-free methods are necessary. Within model-free methods, there are two main optimization types: *on-policy* learning and *off-policy* learning. The main difference between these optimization methods lies in how the agent learns. In on-policy learning, the agent learns and explores the value of the policy as it acts upon the policy. However, in off-policy learning, the agent learns as if picked the best action, while taking a more exploratory action according to a separate behavior policy. These methods can both prove to be useful in solving MDPs, while providing their own benefits and challenges, which will be explored in this report.

A. Frozen Lake MDP

While model-free methods can be used to solve complex and challenging MDPs where the optimal policy may not be easily found, this report explores the policies within the scope of a small, tabular MDP. By implementing these methods on an MDP where the optimal policy is known, it is easier to understand the strengths and shortcomings of each method.

The Frozen Lake MDP used is a simple, "tabular" MDP with a small state and action space. In this MDP, a player must "cross a frozen lake from start to goal without falling into any holes by walking over the frozen lake" [1]. On this lake, there are three types of states: frozen, holes, and a goal state. The latter two of these states are terminal and end a simulation. This lake can be visualized as a small 4x4 grid, as shown by Figure 1 below. In each instance of the simulation, the player will always start in the upper left state at (0,0).

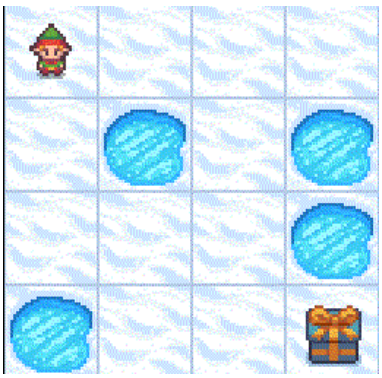


Fig. 1 Visualization of Frozen Lake MDP

*Department of Aerospace Engineering, AIAA Student Member.

From each tile seen on the board above, there are four actions a player can take: moving left, down, right, or up. It is important to note that it is not possible to move off the observable lake, so if an action that would move the player off the lake is taken, the player will remain in the same state.

The goal of this MDP is to reach the goal tile without falling into a hole. A reward is only given for reaching the goal, and the player does not receive a reward for landing on a "frozen" tile.

One important setting within this MDP is the "slipperiness" of the lake. When the lake is not slippery, the actions taken by the agent are deterministic. However, when the lake is slippery, these actions are now stochastic, with a $\frac{1}{3}$ probability of moving in the intended direction or to either of the other tiles perpendicular to the desired movement [1].

B. Objectives

The objective of this paper is evaluate the validity and performance of three model-free optimization methods: on-policy first-visit Monte Carlo control, SARSA, and Q-Learning. By implementing these methods to solve the Frozen Lake MDP, each method will be evaluated on its ability to find an optimal policy, the computational power required for each method, and the behavior of each method within optimization.

II. Methods

A. MDP Set Up

To implement the necessary optimization methods, the Frozen Lake MDP was first fully defined using the *gymnasium* environment. The *gymnasium* environment allows for each implemented method to actually learn the optimal policy through simulations without the need to explicitly define a transition and reward function. Within the gymnasium environment, the transition function and reward function for a given state can be called upon within an algorithm.

B. Monte Carlo Optimization

The first model-free method implemented was on-policy first-visit Monte Carlo (MC), which optimizes the policy, π, by estimating the optimal action-value function $Q(s, a)$ using the roll-out of an entire episode generated using the current policy. It is considered a model-free method because it learns using a sampled experience of actions it can take while interacting with an environment.

1. Algorithm

The algorithm used to implement MC control follows the pseudo code provided by Sutton and Barto's *Reinforcement Learning: An Introduction* [2]. To estimate the optimal action-value function, the action-value function, Q, is first arbitrarily initialized. Additionally, the initial policy, π, is arbitrarily chosen as an ϵ-soft policy in which every action taken by the policy has a non-zero chance of being chosen.

Using this initialized Q and π, a complete episode is then generated following the current policy. This episode can be described as a "sample path" across the frozen lake for this report. After the complete episode is generated, Monte Carlo updates are performed. First, the total return of the episode is found, which is computed as

$$G_t = \gamma G + R_{t+1} \tag{1}$$

where G_t is the return and γ is the discount factor. This return is computed backwards (from $t = T - 1, T - 2, ..., 0$). This return is computed backward to correctly apply the discount factor, which discounts future rewards to encourage the agent to reach the goal state more quickly.

In "first-visit" MC control, the returns, G, are only updated when the agent visits a unique state and action space, $\mathcal{S}_t$ and $\mathcal{A}_t$. For the agent's first visit, the action-value function is updated using the average of the returns for the given state and action pair, $Q(s_t, a_t)$.

To improve the policy after each episode, ε-greedy policy improvement is used to update the policy, allowing the algorithm to favor actions that yield higher expected returns. This improvement can be shown as follows:

$$\pi(a \mid s_t) = \begin{cases} 1 - \varepsilon + \dfrac{\varepsilon}{|\mathcal{A}(s_t)|}, & \text{if } a = A^* \\ \dfrac{\varepsilon}{|\mathcal{A}(s_t)|}, & \text{otherwise} \end{cases} \tag{2}$$

This episode roll-out and update is repeated for many episodes and will converge towards the optimal action-value function $Q^*(s, a)$ because ε-greedy is greedy in the limit with infinite exploration, meaning the agent never stops exploring and will not simply take just the "greedy" action.

C. SARSA

Another model-free method used was SARSA, which is an on-policy method in which the method learns using the current policy to explore the environment. SARSA stands for "state, action, reward, state, action," the steps the method uses to learn the optimal policy. SARSA initializes the current state and selects an action, then receives a reward and moves to a next state based on the chosen action, before selecting a new action after a policy improvement step. Like Monte Carlo, SARSA uses sampling to learn an optimal policy. However, SARSA does not require a complete episode roll-out to update its optimal action-value function; instead it bootstraps and updates the optimal action-value incrementally after each time step. The bootstrapping occurs from using a current estimate of the return to update $Q(s, a)$ after each time step, not a complete episode.

1. Algorithm

The algorithm used to implement SARSA follows the pseudo code provided by Sutton and Barto's *Reinforcement Learning: An Introduction* [2]. To estimate the optimal action-value function, the action-value function, Q, is first arbitrarily initialized. Unlike Monte Carlo, there is no policy being carried throughout the SARSA algorithm, so there is no need to initialize an arbitrary policy.

An episode is then created using the initialized Q, where the chosen action is derived from an ε-greedy policy. For this model, this policy can be shown as follows:

$$a = \begin{cases} \text{random action from } \mathcal{A}(s), & \text{with probability } \varepsilon \\ \arg\max\limits_{a} Q(s, a), & \text{with probability } 1 - \varepsilon \end{cases} \tag{3}$$

Using this action, the next state is then found using the Frozen Lake environment, and the next action is chosen using the same function derived from ε-greedy.

Then, as discussed previously, a temporal difference method is used to update $Q(s, a)$ after each time step. This method can be shown as follows:

$$Q(S, A) \leftarrow Q(S, A) + \alpha\big[R + \gamma Q(S', A') - Q(S, A)\big] \tag{4}$$

In this update, the term $R + \gamma Q(S', A')$ can be considered the temporal difference target, and the $R + \gamma Q(S', A') - Q(S, A)$ term is the temporal difference error. In this method, α is the learning rate, and a parameter that determines how the method will seek the optimal policy. This update is repeated for every time step in many episodes.

D. Q-Learning

Q-Learning is the final model-free method used to find the optimal policy is essentially the same method as SARSA, except it uses *off-policy* learning, different than SARSA's *on-policy* learning. This allows the method to update the action-value function, $Q(s, a)$, with the best possible action at each state, not the action it is taking.

1. Algorithm

The algorithm used to implement Q-Learning follows the pseudo code provided by Sutton and Barto's *Reinforcement Learning: An Introduction* [2]. This algorithm is identical to that of SARSA, with the only difference being in the

$Q(s, a)$ update that occurs after each time step. Because Q-Learning selects the best possible action, the temporal difference target becomes:

$$TD_{Target} = R + \gamma \max_a Q(S', a) \tag{5}$$

Like SARSA, this update is performed after every time step for many episodes.

E. Algorithm Code Implementation

To implement these algorithms into code, the *gymnasium* environment is used to easily compute the next state and reward for a given action within the Frozen Lake environment.

Although each algorithm is supposed to run infinitely until convergence, this was not feasible due to local computing power. Instead, each method was run for 20,000 episodes, which provided ample time for the algorithms to explore the environment without requiring significant computing power.

An additional argument added to each algorithm was an optional ε-decay, which decays ε linearly from a value of 0.3 to 0.05 across the 20,000 episodes, allowing for exploration to be minimized as the agent learns more about the environment.

Because each algorithm provides an optimal action-value function, $Q(s, a)$, at the end of its run, to evaluate these functions, it was necessary to find the policy from this $Q(s, a)$. To do this, a greedy policy is extracted from $Q(s, a)$ using: $\pi(s) = \arg\max_a Q(s, a)$.

F. Evaluation Returns

To evaluate each method, it was necessary to provide a metric to measure and track each method over its run of 20,000 episodes. To do this, the evaluation return method provided in Albrecht's *Multi-Agent Reinforcement Learning: Foundations and Modern Approaches* was used [3]. In each method, the greedy policy is intermittently extracted from the action-value function $Q(s, a)$, and used to run a set of individual episodes following that policy. The average return is then computed for this set of episodes, which is the "evaluation return" for that time step.

In this report, these evaluation returns were performed every 2,000 time steps, and 20 episodes were run to compute each evaluation return. These parameters were selected due to computing power limitations, as more advanced methods may evaluate more episodes and more often.

III. Results

To evaluate the validity and success of each method, each method was run with and without ε-decay for both the slippery and non-slippery conditions. The learning rate, α, was also varied to measure its effectiveness. For each run, the evaluation return was plotted to compare and evaluate the methods

A. Non-Slippery vs. Slippery Condition

To determine a baseline for evaluation, all three methods were run with a constant $\varepsilon = 0.3$ and a learning rate of $\alpha = 0.5$. An aggressive learning rate was selected to ensure convergence within the limited number of episodes, as computing power limited the number of simulations ran. The evaluation returns for all three methods for both the non-slippery and slippery conditions are shown below in Figures 2 - 3.

As can be seen in these evaluation return plots, all three methods had an upward trend for the deterministic non-slippery condition, indicating convergence. Both the MC and Q-Learning methods found an optimal policy relatively quickly. However, SARSA found an optimal policy, but due to exploration, kept oscillating between an optimal policy and zero reward. Monte Carlo is able to maintain its optimal policy throughout the entirety of the simulation because it carries its policy throughout the simulation, it does not take random actions based on ε-greedy behavior. Q-Learning is able to hold its convergence because it always computes the expected return based on the best possible action, so the action-value function will, by nature, maintain a policy that results in a reward. SARSA, on the other hand, still has exploration even once it finds an optimal policy, so it will continue to explore and update the action-value function with non-optimal policies. Because the non-slippery condition is deterministic, a policy will always yield a reward of 0 or 1, which is what causes the oscillation between these values seen by the SARSA curve.

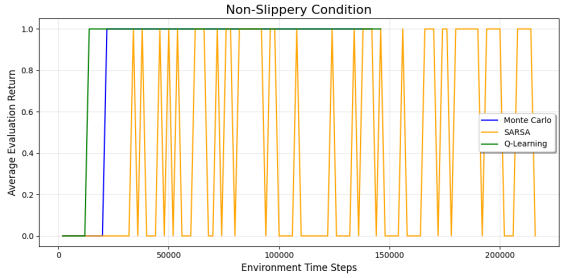


Fig. 2 Evaluation Return vs Environment Time Steps for Non-Slippery Condition

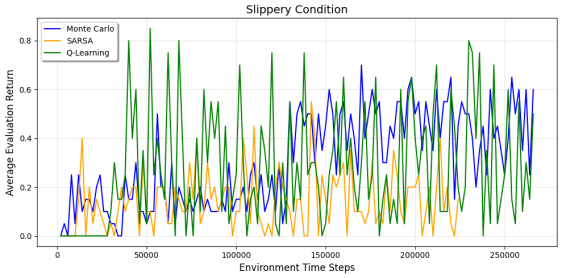


Fig. 3 Evaluation Return vs Environment Time Steps for Slippery Condition

However, in the stochastic environment of the slippery condition, a policy may sometimes yield a reward of 0 and sometimes 1. This is what causes the much more jagged shape of the plot in Figure 3. In the stochastic version, no policy can guarantee a return of 1, as there is always an element of randomness due to the stochastic nature. However, each method yields an upward trend, indicating convergence towards the true policy. It is seen in the plot that at the beginning, Q-Learning yields the best policy; however, as time continues, MC finds a more stable policy. SARSA is again the least convergent, likely due to its exploration. The exploration of Q-Learning likely also causes the high variance seen. Because MC carries the policy throughout, a more stable policy is found.

B. Epsilon Decay

To evaluate the impact of the ε value on the policies, ε-decay was applied to each of the functions with the same learning rate of $\alpha = 0.5$. The evaluation returns with ε-decay are shown below in Figures 4-5.

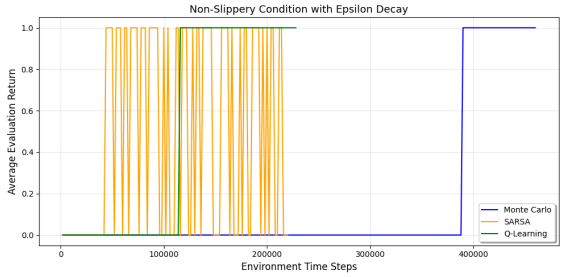


Fig. 4 Evaluation Return with ε-Decay for Non-Slippery Condition

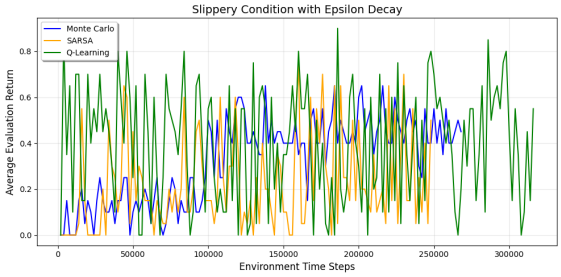


Fig. 5 Evaluation Return with ε-Decay for Slippery Condition

From the above figures, it can be seen that decaying ε actually had a negative effect in the non-slippery environment. Although all three methods still converged, they required more time steps and computational power to do so. This is especially true for MC, which took over 10 times the time steps to converge. In a small MDP, reducing exploration may keep the agent trapped with a non-optimal policy and rely on a small probability of exploring outside of this policy, which may increase the time necessary for convergence.

In the stochastic, slippery environment, there is some improvement seen with SARSA and Q-Learning, but MC converges to a policy with a lower evaluation return than seen with no ε-decay. Because the policy is carried through in MC, less exploration can prevent the method from finding a more optimal policy. However, in methods where the action is randomly chosen such as SARSA and Q-Learning, less exploration can help focus on policies that have yielded better results.

C. Lower Learning Rate

To evaluate the impact of the learning rate on the policies, a lower learning rate of $\alpha = 0.2$ was applied to each of the policies with a constant ε of 0.3. The evaluation returns with a lower α are shown below in Figures 6-7.

It can be seen above that the learning rate does not have an effect in deterministic environments, as there is no

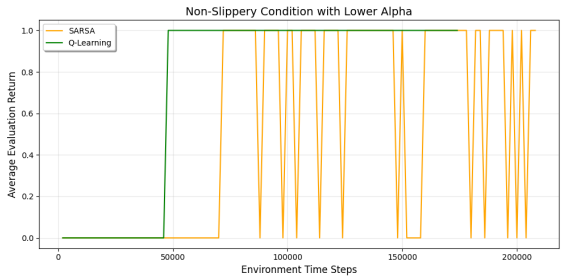


Fig. 6 Evaluation Return with Lower Learning Rate for Non-Slippery Condition

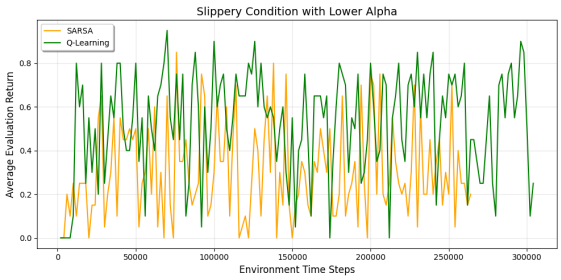


Fig. 7 Evaluation Return with Lower Learning Rate for Slippery Condition

random action that alpha may have a sensitivity towards. In the equation for the temporal difference target, alpha affects how much the newly learned information has an effect on $Q(s,a)$. In a deterministic environment, this new information is replicable in every simulation, so the weight placed on this information does not matter. However, in stochastic environments, a lower alpha can make a method take longer to converge due to the fact that the new information is not affecting $Q(s,a)$ as much, which is why Q-Learning and SARSA appear stagnant. In a situation where the number of episodes is limited, a higher α is desired to ensure the method can learn.

D. Policy

A sample policy in the stochastic environment is shown below in Figures 8-9, showing two policies that have a similar evaluation return, but different policies due to the stochastic nature, illustrating how different methods may yield different policies if they are not derived from the true optimal value function $V^*(S)$. These policies are from MC and Q-Learning for the baseline case.

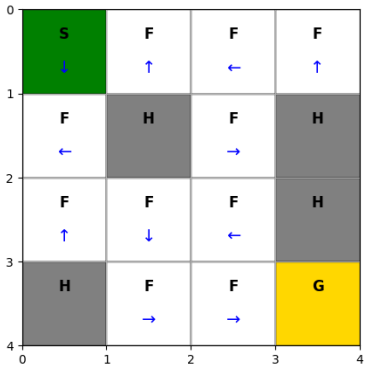


Fig. 8 MC Policy for Slippery Condition

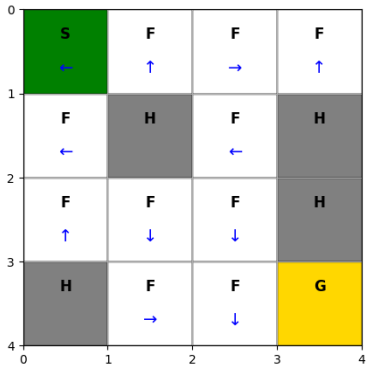


Fig. 9 Q-Learning Policy for Slippery Condition

IV. Conclusions

In conclusion, this report provided valuable insight and understanding into the theory and convergence of model-free methods used to find optimal policies in MDPs. Policies such as on-policy Monte Carlo that carry a policy through can converge more stably than methods that rely on ε-greedy actions such as SARSA and Q-Learning. Epsilon decay may be a valuable tool in finding optimal policies, but it may also prevent exploration and the discovery of an optimal policy when only one optimal policy exists. In deterministic environments, the learning rate is not as impactful as in stochastic environments, due to the way the action-value function is updated. This report also provided a meaningful way to evaluate a policy with evaluation returns, and showed that even with similar evaluation returns, two policies may be vastly different. This report provided a foundation for solving more complex MDPs and understanding sources of error that may arise in using model-free methods to optimize these MDPs.

References

[1] Foundation, F., "Frozen Lake," *Gymnasium Documentation*, 2025. URL `https://gymnasium.farama.org/environments/toy_text/frozen_lake/`.

[2] Sutton, R. S., and Barto, A. G., *Reinforcement Learning: An Introduction*, 2nd ed., MIT Press, Cambridge, MA, 2018.

[3] Albrecht, S. V., Christianos, F., and Schäfer, L., *Multi-Agent Reinforcement Learning: Foundations and Modern Approaches*, MIT Press, 2024.